

```

#include <iostream>

#include <vector>

#include <queue>

using namespace std;


// DFS Function

void DFS(int node, vector<vector<int>> &graph, vector<bool> &visited)
{
    visited[node] = true;

    cout << node << " ";


    for (int next : graph[node])
    {
        if (!visited[next])
            DFS(next, graph, visited);
    }
}


// BFS Function

void BFS(int start, vector<vector<int>> &graph, int V)
{
    vector<bool> visited(V, false);

    queue<int> q;


    visited[start] = true;
    q.push(start);


    while (!q.empty())
    {
        int node = q.front();
        q.pop();
    }
}

```

```

        cout << node << " ";

    for (int next : graph[node])
    {
        if (!visited[next])
        {
            visited[next] = true;
            q.push(next);
        }
    }
}

int main()
{
    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    cout << "Enter number of edges: ";
    cin >> E;

    vector<vector<int>> graph(V);

    cout << "Enter edges (u v):\n";
    for (int i = 0; i < E; i++)
    {
        int u, v;
        cin >> u >> v;
    }
}

```

```

graph[u].push_back(v);
graph[v].push_back(u); // Remove for directed graph
}

int start;
cout << "Enter starting vertex: ";
cin >> start;

vector<bool> visited(V, false);

cout << "\nDFS Traversal: ";
DFS(start, graph, visited);

cout << "\nBFS Traversal: ";
BFS(start, graph, V);

return 0;
}

```

OUTPUT:

Enter number of vertices and edges: 8 4

Enter edges:

2 3

4 1

5 2

3 4

Enter starting vertex: 4

DFS Traversal: 4 1 3 2 5

BFS Traversal: 4 1 3 2 5

Execution Time:

DFS: 23550 ns

BFS: 9900 ns

=== Code Execution Successful ===